WEB SCRAPING USING PYTHON & SELENIUM

Python Web Scraping and Data Collection

Prepared By: Ritesh Chaudhary

Contents

1. Project Overview 2

1.1 Introduction 2

1.2 Project Objectives 2

1.3 Project Scope 2

2. Tools and Technologies 3

3. Target Website and Data Structure 3

3.1 Target Website 3

3.2 Data Fields 3

3.3 Sample Extracted Data 3

4. Program Implementation 4

4.1 Installing and Importing Libraries 4

4.2 Browser Initialization and Website Access 4

4.3 Locating HTML Table Elements 4

4.4 Extracting Information 5

4.5 Organizing Extracted Data 6

4.6 Creating a Python Dictionary 6

4.7 Closing the Browser Session 7

5. Data Processing and Storage Using Pandas 7

5.1 Converting Dictionary into a DataFrame 7

5.2 Exporting Data into CSV Format 7

5.3 Exporting Data into JSON Format 8

5.4 Exporting Data into Excel Format 8

6. Program Workflow 8

7. Results and Output 9

8. Limitations 9

9. Conclusion 10

1. Project Overview

1.1 Introduction

Web scraping is the process of automatically collecting information from websites using software programs. It reduces the need for manually browsing websites and copying information by automating the process of accessing web pages, locating HTML elements, extracting data, and organizing the collected information.

This project demonstrates web scraping using Python and Selenium. The program automates Google Chrome, accesses a practice website containing a book-information table, and extracts details such as book names, authors, ratings, and prices.

The collected information is organized into Python lists and a dictionary before being converted into a Pandas DataFrame. The final dataset is exported into CSV, JSON, and Excel formats for further use.

1.2 Project Objectives

The main objectives of this project are:

- Understand the basic concepts of web scraping and browser automation.
- Automate Google Chrome using Python and Selenium.
- Identify and locate HTML table elements using Selenium locators.
- Extract structured information from HTML table rows and columns.
- Organize extracted information using Python lists and dictionaries.
- Convert collected data into a Pandas DataFrame.
- Export and retrieve datasets in CSV, JSON, and Excel formats.

1.3 Project Scope

The project focuses on extracting structured book information from a publicly accessible practice webpage.

The program performs the following tasks:

1. Opens the target website using Selenium.
2. Locates the HTML table and retrieves its rows.
3. Extracts the text from individual table cells.
4. Stores the extracted information in separate Python lists.
5. Combines the lists into a Python dictionary.
6. Converts the dictionary into a Pandas DataFrame.
7. Exports the dataset into CSV, JSON, and Excel files.
8. Reads the exported files back into Pandas DataFrames to verify the stored information.

2. Tools and Technologies

The following tools and libraries are used in this project.

Technology	Purpose
Python	Main programming language used to develop the scraper.
Selenium	Automates the web browser and locates HTML elements.
Pandas	Organizes extracted data into DataFrames and handles file operations.
Google Chrome	Browser controlled by Selenium to access the target website.
CSV	Stores structured data in comma-separated format.
JSON	Stores structured data in JavaScript Object Notation format.
Excel	Stores structured data in spreadsheet format.

3. Target Website and Data Structure

3.1 Target Website

Website: https://tryscrapeme.com/web-scraping-practice/beginner/abc

The website provides a practice table containing book-related information. The table consists of four columns and ten records.

3.2 Data Fields

The program extracts the following information:

Field	Description
Name	Name or title of the book.
Author	Name of the book's author.
Stars	Rating assigned to the book.
Price	Price of the book.

3.3 Sample Extracted Data

The following table presents a sample of the information collected by the program.

Name	Author	Stars	Price
Clean Code	Robert C. Martin		26.56
The Legend of Zelda	Shigeru Miyamoto		27.56
Don't Make Me Think, Revisited	Steve Krug		23.03

Godel, Escher, Bach	Douglas R. Hofstadter		16.37
Superintelligence	Nick Bostrom		9.93

Total records collected: 10
Total fields: 4

4. Program Implementation

4.1 Installing and Importing Libraries

Selenium is installed using pip:

pip install selenium

The required Selenium modules are imported to initialize the browser and locate webpage elements.

from selenium import webdriver
from selenium.webdriver.common.by import By

Pandas is imported later to organize and export the collected information.

import pandas as pd

4.2 Browser Initialization and Website Access

The program initializes a Chrome browser using Selenium WebDriver and navigates to the target website.

website = "https://tryscrapeme.com/web-scraping-practice/beginner/abc"

driver = webdriver.Chrome()
driver.get(website)

Here:

- webdriver.Chrome() initializes the Chrome browser.
- driver.get(website) navigates to the specified webpage.

4.3 Locating HTML Table Elements

The target webpage stores the book information inside an HTML table.

The program uses Selenium's find_element() and find_elements() methods to locate the table body and its rows.

table_body = driver.find_element(By.TAG_NAME, 'tbody')

rows = table_body.find_elements(By.TAG_NAME, 'tr')

The following HTML structure illustrates how the webpage organizes its table data:

```
<table>
  <thead>
    <tr>
      <th>Heading 1</th>
      <th>Heading 2</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>Data 1</td>
      <td>Data 2</td>
    </tr>
  </tbody>
</table>
```

In this structure:

- <table> represents the complete table.
- <thead> contains the table headings.
- <tbody> contains the actual records.
- <tr> represents an individual row.
- <td> represents an individual data cell.

4.4 Extracting Information

After locating the table rows, the program iterates through each row and identifies its columns.

The .text property retrieves the visible text from each HTML element.

```
for row in rows:

  columns = row.find_elements(By.TAG_NAME, 'td')

  for col in columns:
    print(col.text)
```

```
    print()
```

This extracts the book names, authors, ratings, and prices from the webpage.

4.5 Organizing Extracted Data

To organize the extracted information, four separate Python lists are created.

```
name_col = []
author_col = []
stars_col = []
price_col = []
```

The program then iterates through the table rows and stores each column's information in its corresponding list.

```
for row in rows:

    columns = row.find_elements(By.TAG_NAME, 'td')

    name_col.append(columns[0].text)
    author_col.append(columns[1].text)
    stars_col.append(columns[2].text)
    price_col.append(columns[3].text)

print('Data Stored Successfully.')
```

Each list represents a separate field in the dataset.

4.6 Creating a Python Dictionary

After collecting the information, the program combines the four lists into a dictionary.

```
data = {
    "Name": name_col,
    "Author": author_col,
    "Stars": stars_col,
    "Price": price_col
}
```

This organizes the collected information into a structured key-value format, making it easier to convert into a DataFrame.

4.7 Closing the Browser Session

After extracting the required information, the browser session is closed.

driver.quit()

This releases the browser resources used during the scraping process.

5. Data Processing and Storage Using Pandas

5.1 Converting Dictionary into a DataFrame

Pandas is used to convert the extracted dictionary into a DataFrame.

import pandas as pd

df = pd.DataFrame(data)

df

The DataFrame organizes the collected information into rows and columns, making it easier to view, process, and export.

The resulting dataset contains ten records and four columns:

- Name
- Author
- Stars
- Price

5.2 Exporting Data into CSV Format

The DataFrame is exported into a CSV file using the to_csv() method.

df.to_csv('file.csv', index=False)

The index=False parameter prevents Pandas from adding an additional index column to the exported file.

The CSV file is then read back using:

csv_df = pd.read_csv('file.csv')

csv_df

5.3 Exporting Data into JSON Format

The dataset is exported into JSON format using the to_json() method.

df.to_json('file.json', indent=4)

The indent=4 parameter formats the JSON output with indentation for better readability.

The exported JSON file is read back using:

json_df = pd.read_json('file.json')

json_df

5.4 Exporting Data into Excel Format

The DataFrame is exported into an Excel spreadsheet using:

df.to_excel('file.xlsx', index=False)

The exported Excel file is read back using:

excel_df = pd.read_excel('file.xlsx')

excel_df

These operations demonstrate how the collected dataset can be stored and retrieved in three commonly used data formats.

6. Program Workflow

The overall workflow of the project is presented below.

Step 1: Browser Automation
Python initializes Google Chrome using Selenium WebDriver.

Step 2: Website Navigation
Selenium opens the target website containing the book-information table.

Step 3: Element Identification
The program locates the HTML table body, rows, and individual columns.

Step 4: Data Extraction
The .text property retrieves the required information from each table cell.

Step 5: Data Organization
Extracted information is stored in Python lists and combined into a dictionary.

Step 6: DataFrame Creation
Pandas converts the dictionary into a structured DataFrame.

Step 7: Data Export
The dataset is exported into CSV, JSON, and Excel formats.

Step 8: Data Verification
The exported files are read back into Pandas DataFrames to check the stored information.

7. Results and Output

The program successfully demonstrates the extraction and storage of book information from the target website.

The final dataset contains:

Output	Result
Records extracted	10
Columns extracted	4
Data structure	Python Dictionary
Data processing	Pandas DataFrame
CSV output	file.csv
JSON output	file.json
Excel output	file.xlsx

The extracted information is available in a structured format and can be reused for further data processing, reporting, or analysis.

The CSV, JSON, and Excel read operations also demonstrate that the exported datasets can be retrieved and displayed successfully.

8. Limitations

The project has the following limitations:

- The scraper is designed for the specific HTML table structure of the selected website.
- Changes to the webpage structure may require modifications to the Selenium locators.
- The program depends on internet connectivity and access to the target website.
- The current implementation extracts data from a single webpage.
- The program demonstrates data collection and storage rather than advanced data cleaning, statistical analysis, or visualization.

- The collected data represents the information available on the website when the scraping operation is performed.

9. Conclusion

This project demonstrates the practical application of Python and Selenium for automated web scraping and structured data collection.

Using Selenium WebDriver, the program automates Google Chrome, accesses a target webpage, identifies HTML table elements, and extracts book-related information. The extracted records are organized using Python lists and dictionaries before being converted into a Pandas DataFrame.

The project also demonstrates how collected information can be exported and retrieved using CSV, JSON, and Excel formats.

Through this implementation, the project provides practical experience with browser automation, HTML element identification, data extraction, Python data structures, and Pandas-based data storage. These skills are relevant to web data collection, data preparation, and software automation tasks.

End of Project Documentation